

Projet R

Lucien LE GALL, Haobin JIANG

Objectif : Étudier la tendance et la saisonnalité d'une série chronologique représentant le nombre de voyageur sur un réseau de transport à Chicago puis fournir une prévision avec une certaine erreur relative pour les 12 prochains mois. Les données seront moyennées par mois.

1 Mises en forme des données et description de la source

Les données représentent la fréquentation journalière de transport en commun entre le 1er janvier 2001 et le 12 décembre 2020 à Chicago. La première étape consiste à lire les données :

```
data <- read.csv("CTA_-_Ridership_-_Daily_Boarding_Totals.csv", header = TRUE,
  stringsAsFactors = FALSE)
```

Puis l'on accède au nom de chaque colonne de la façon suivante :

```
> print(names(data))
[1] "service_date" "day_type" "bus"
[4] "rail_boardings" "total_rides"
```

On a donc 5 colonnes au total : date, type de jour, numéro du bus, embarquement ferroviaire et nombre total de courses.

La moyenne par mois a été tracée :

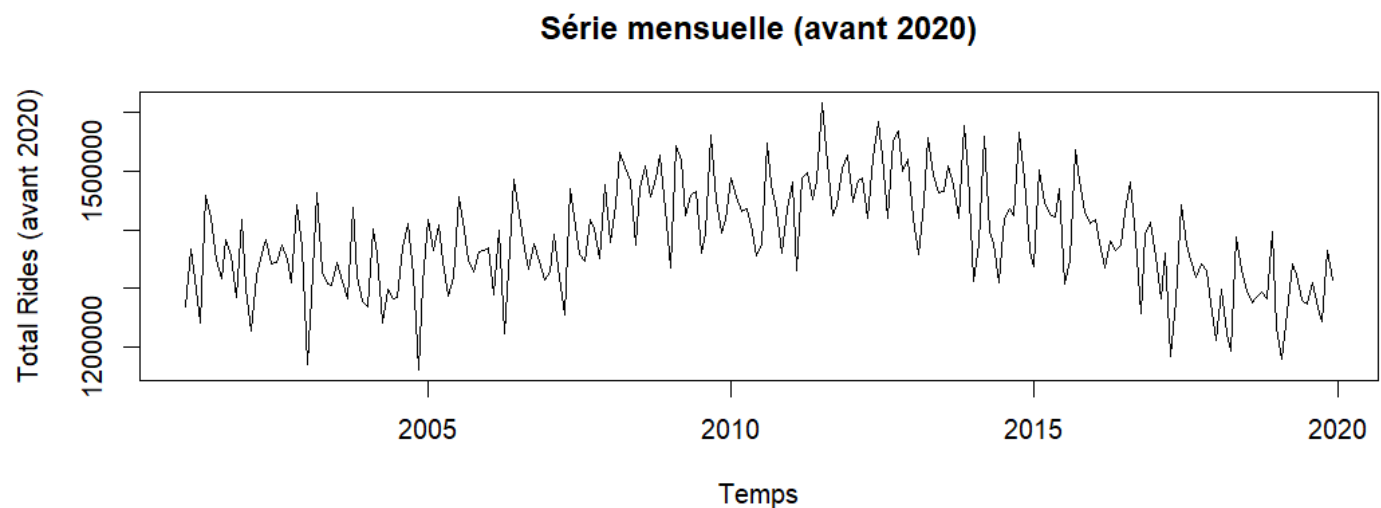


Figure 1: Valeur moyenne de la fréquentation par mois

2 Analyse descriptive

Pour l'analyse descriptive des données, nous avons utilisé la fonction **boxplot**

```
boxplot(as.numeric(monthly_ts), main = "Boxplot mensuel", ylab = "Total Rides")
hist(monthly_ts, main = "Histogramme mensuel", xlab = "Total Rides")
```

Les résultats sont les suivants :

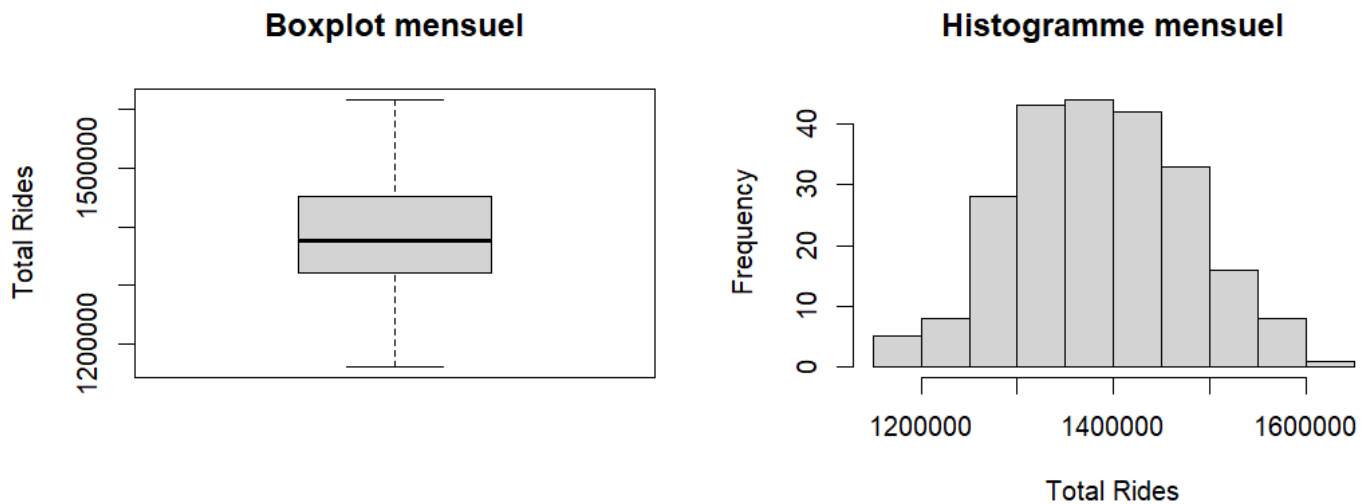


Figure 2: Boxplot et histogramme

À l'aide de la commande `summary(monthly_xts)`, il est possible de retrouver les valeurs affichées par `boxplot` pour les valeurs moyennes par mois :

Statistique	total_rides
Minimum	1160237
1 ^e Quartile	1321619
Médiane	1376820
Moyenne	1386809
3 ^e Quartile	1421743
Maximum	1617359

Table 1: Statistiques descriptives des trajets (total_rides)

Enfin, il est possible de visualiser l'autocorrélogramme de la série avant d'étudier la tendance et la saisonnalité pour se donner une idée de sa stationnarité. Clairement, elle n'est pas stationnaire.

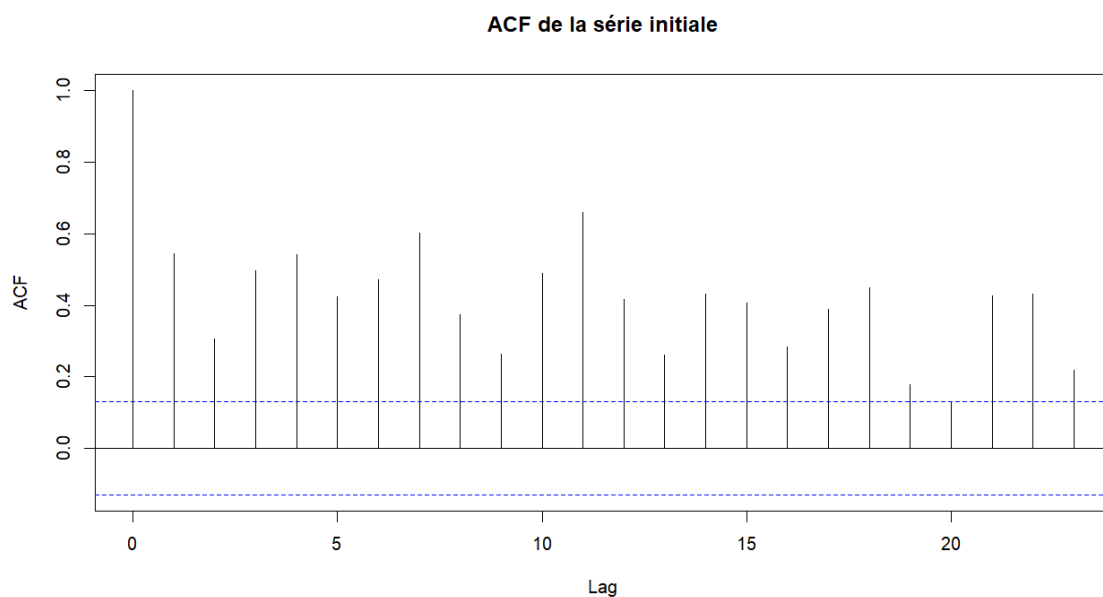


Figure 3: ACF série temporelle moyennée par mois

3 Analyse de la tendance

3.1 Estimation paramétrique de la tendance

La régression linéaire avec un modèle linéaire (**lm**) donne le résultat suivant sur les données moyennées par mois :

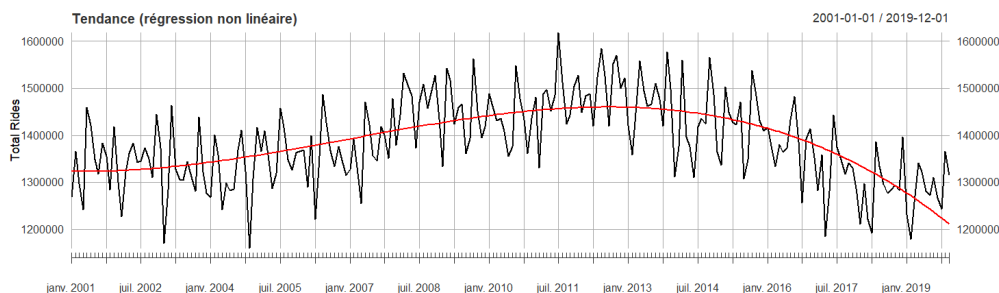


Figure 4: Estimation de la tendance par régression linéaire

L'autocorrélogramme (ACF) et les résidus après estimation de la tendance donnent :

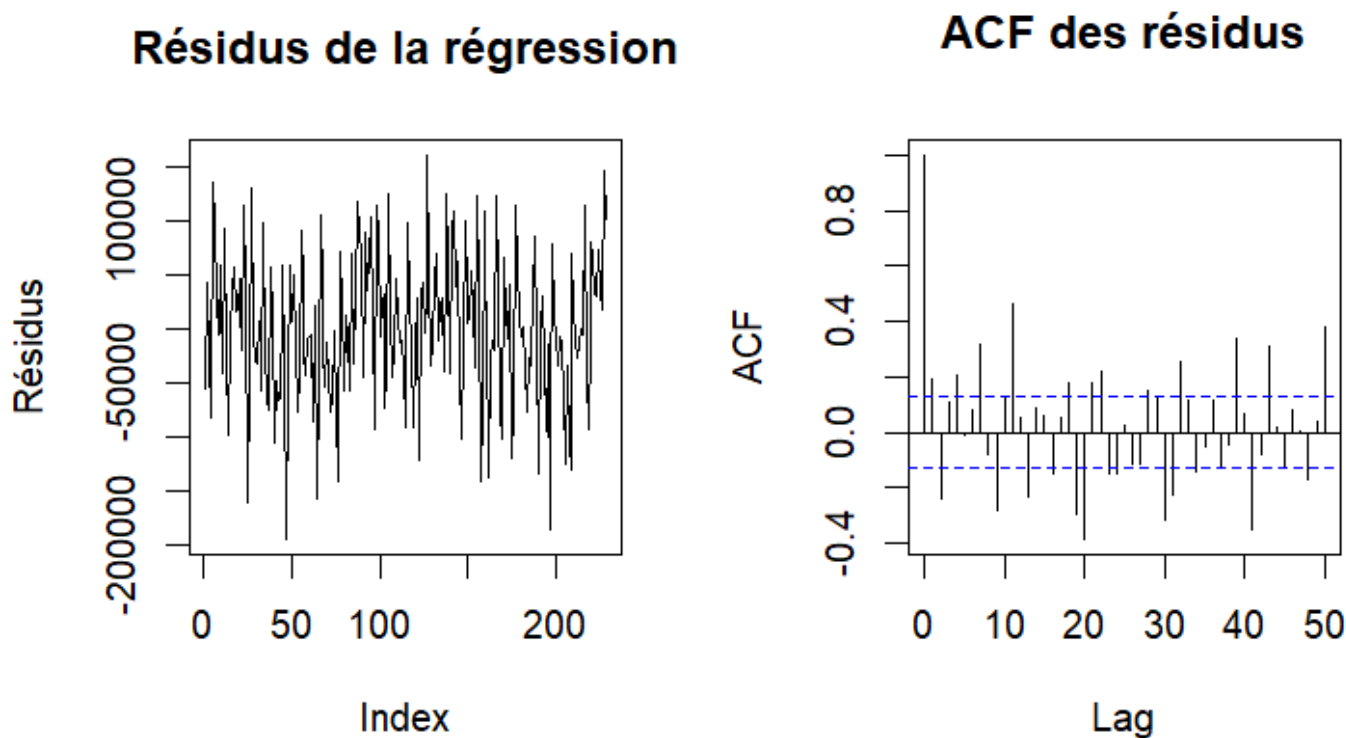


Figure 5: Résidus et ACF

Les résidus sont relativement faibles tout comme les autocorrélations, on s'approche donc d'une série stationnaire. Par la suite d'autres méthodes sont implémentées.

En outre, nous avons également analysé l'évolution de la fréquentation des bus sur les mêmes intervalles de temps et nous l'avons comparée à l'évolution de la fréquentation des transports publics.

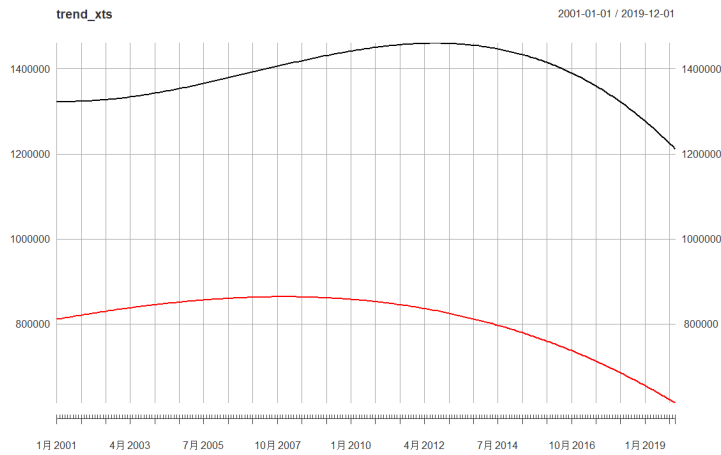


Figure 6: Fréquentation totale vs fréquentation des bus(rouge)

Nous avons constaté une nette tendance à la hausse de la fréquentation totale autour de 2006, mais peu de changement dans la fréquentation des bus. La raison en est qu'il y a eu une augmentation significative de la fréquentation du métro dans la ville de Chicago autour de 2006. La même année, la Loop-Pink Line de Chicago a été inaugurée

3.2 Estimation par moyenne mobile

Le résultat est le suivant :

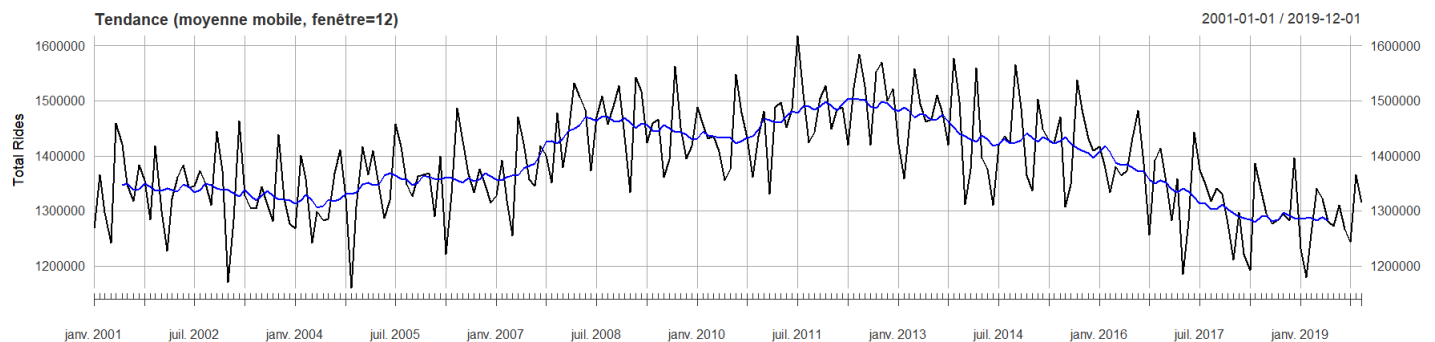


Figure 7: Estimation de la tendance par moyenne mobile

```
ma_window <- 12
ma_trend <- stats::filter(monthly_ts, filter = rep(1/ma_window, ma_window),
                          method = "convolution", sides = 2, circular = TRUE)
ma_trend_xts <- xts(as.numeric(ma_trend), order.by = monthly_avg2$Date)
plot(monthly_xts, type = "l", main = "Tendance (moyenne mobile, fenetre=12)",
     xlab = "Temps", ylab = "Total Rides")
lines(ma_trend_xts, col = "blue", lwd=2)
```

La « fenêtre » est fixée à 12 car les données originales sont des moyennes mensuelles, ce qui permet d'obtenir une meilleure tendance sur une base annuelle.

3.3 Estimation par noyau gaussien

Le résultat de l'estimation de la tendance par noyau gaussien est le suivant :

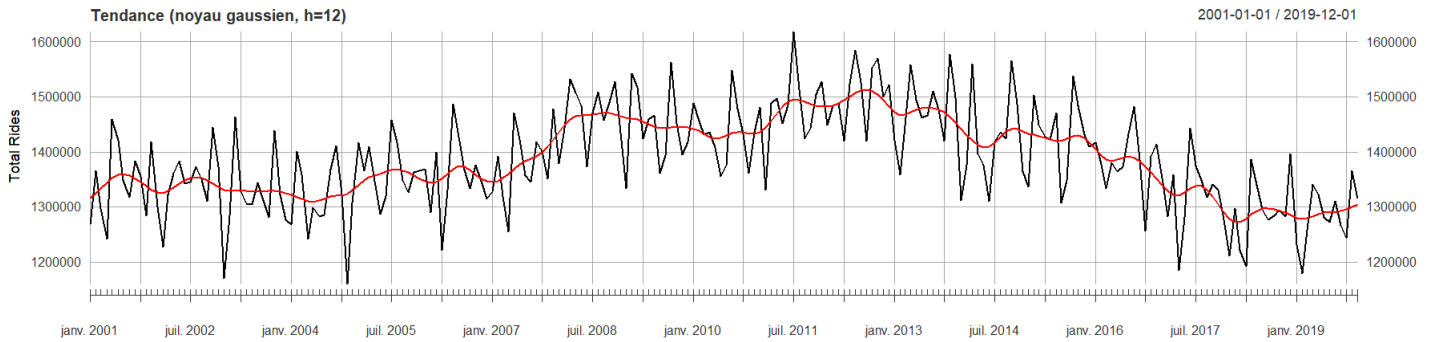


Figure 8: Estimation de la tendance par noyau gaussien

```
h <- 12
x_seq <- seq(1, n, length.out = n)
gauss_kernel <- function(x_val) {
  dnorm(x_val - t, 0, sd = sqrt(h/2)) / sum(dnorm(x_val - t, 0, sd = sqrt(h/2)))
}
W <- matrix(unlist(lapply(x_seq, gauss_kernel)), nrow = n, ncol = n, byrow = FALSE)
kernel_trend <- colSums(as.numeric(monthly_ts) * W)
kernel_trend_xts <- xts(kernel_trend, order.by = monthly_avg$Date)
plot(monthly_xts, type = "l", main = "Tendance (noyau gaussien, h=12)",
      xlab = "Temps", ylab = "Total Rides")
lines(kernel_trend_xts, col = "red", lwd = 2)
```

3.4 Estimation par base de splin

Le résultat de l'estimation de la tendance à l'aide de la fonction **gam** donne :

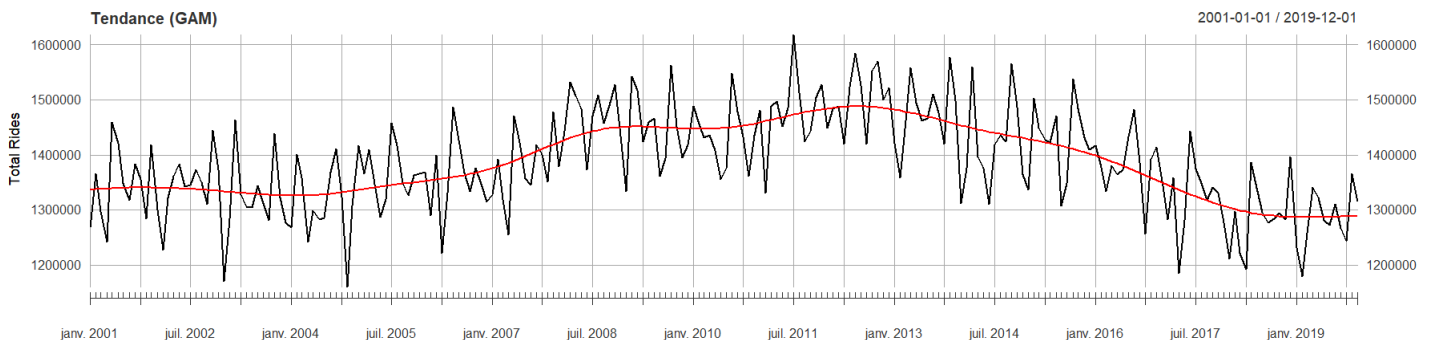


Figure 9: Estimation de la tendance par bases de splin

```
t_numeric <- as.numeric(monthly_avg$Date)
g <- gam(monthly_ts ~ t_numeric2 + s(t_numeric, k = 50))
gam_trend <- g$fitted.values
gam_trend_xts <- xts(as.numeric(gam_trend), order.by = monthly_avg$Date)
plot(monthly_xts, type = "l", main = "Tendance (GAM)",
      xlab = "Temps", ylab = "Total Rides")
lines(gam_trend_xts, col = "red", lwd = 2)
print(summary(g))
```

En ajustant les degrés de liberté, nous pouvons obtenir des ajustements plus lisses.

4 Estimation de la saisonnalité

Avant d'étudier la saisonnalité de la série temporelle, il est nécessaire de retirer la tendance. Après comparaison, nous estimons que la méthode 'gam' s'ajuste plus facilement et peut être utilisée comme référence pour les tendances. ce qui est fait à l'aide d'une régression linéaire dans le code suivant :

```
detrended <- monthly_xts - gam_trend_xts
plot(detrended, type = "l", main = "S r i e  d  t e n d r  e  (d o n n e s  -  t e n d a n c e)",
      xlab = "T e m p s", ylab = "R s i d u s")
acf(as.numeric(detrended), lag.max = 50, main = "A C F  d e  l a  s  r i e  d  t e n d r  e")
```

4.1 Estimation de la saisonnalité par moyenne mobile

Le résultat suivant a été obtenu :

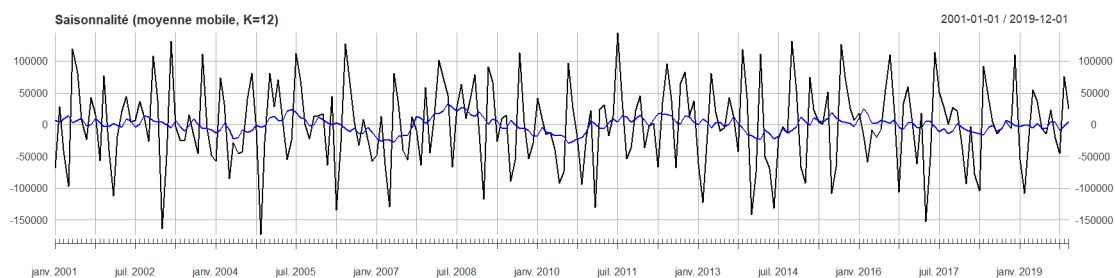


Figure 10: Étude de la saisonnalité par moyenne mobile

Cette méthode ne rend pas bien compte des valeurs extrêmes et des minima des données originales et n'est guère utile pour nos analyses saisonnières.

4.2 Estimation par noyaux tricubes

Pour modéliser les tendances saisonnières de manière plus souple, nous utilisons le noyau de **Losse**.

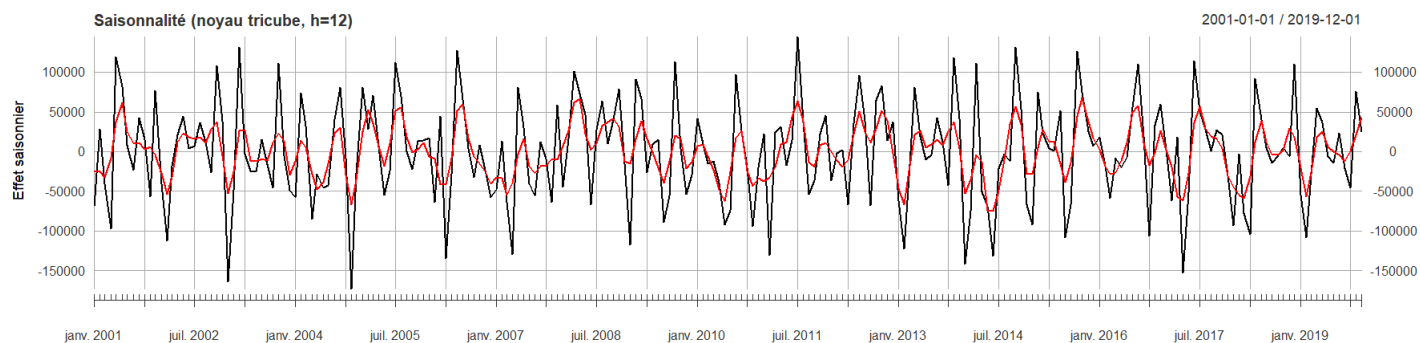


Figure 11: Estimation de la saisonnalité par noyaux tricubes

```
h_season <- 12
x_seq2 <- seq(1, n, length.out = n)
tricube_kernel <- function(u) {
  w <- (1 - abs(u)^3)^3
  w[abs(u) >= 1] <- 0
  return(w)
}
W_season <- matrix(unlist(lapply(x_seq2, function(x_val) {
  u <- (x_val - t) / sqrt(h_season/2)

```

```

w <- tricube_kernel(u)
w / sum(w)
))), ncol = n, nrow = n, byrow = FALSE)
season_kernel <- colSums(as.numeric(detrended) * W_season)
season_kernel_xts <- xts(season_kernel, order.by = monthly_avg$Date)
plot(detrended, type = "l", main = "Saisonnalité (noyau tricube, h=12)",
     xlab = "Temps", ylab = "Effet saisonnier")
lines(season_kernel_xts2, col = "red", lwd = 2)

```

Cependant, les résultats de la simulation ne montrent pas une forte distribution de régression saisonnière et, compte tenu de mes connaissances, je ne pense pas que les résultats puissent être améliorés en ajustant la méthodologie existante.

4.3 Estimation de la saisonnalité par Fourier

Cette méthode fournit de loin la meilleure analyse de la saisonnalité.

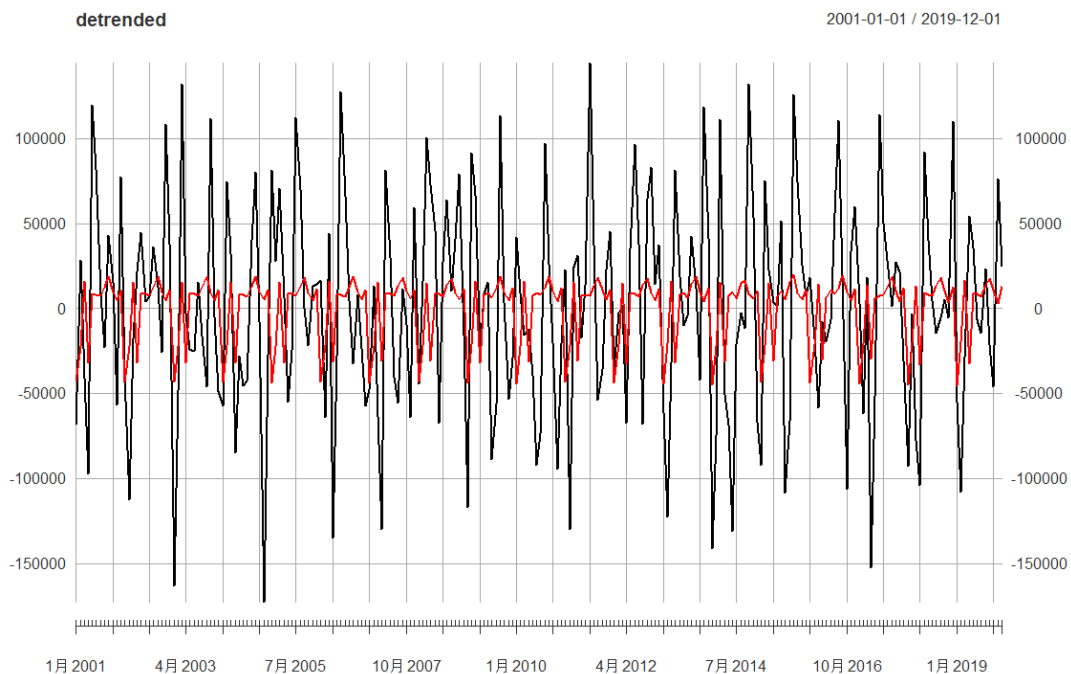


Figure 12: Fourier

```

w=2*pi/12##windows
fourier<-cbind(cos(w*t), sin(w*t))
K<-20
for(i in c(2:K))
{
  fourier<-cbind(fourier,cos(i*w*t), sin(i*w*t))
}
matplot(fourier[,c(1,3)],type='l')
dim(fourier)

r2<- NULL
err <- NULL
for(i in seq(2, ncol(fourier), by=2))
{
  reg<-lm(detrended~fourier[,1:i]-1)
  s <- summary(reg)
  r2 <- c(r2, s$r.squared)
}

```

```

err <- c(err, sum(reg$residuals^2))
}

plot(err, type='b', pch=20, ylim=c(50, 200))
lines(err+2*c(1:ncol(fourier)), col='red')

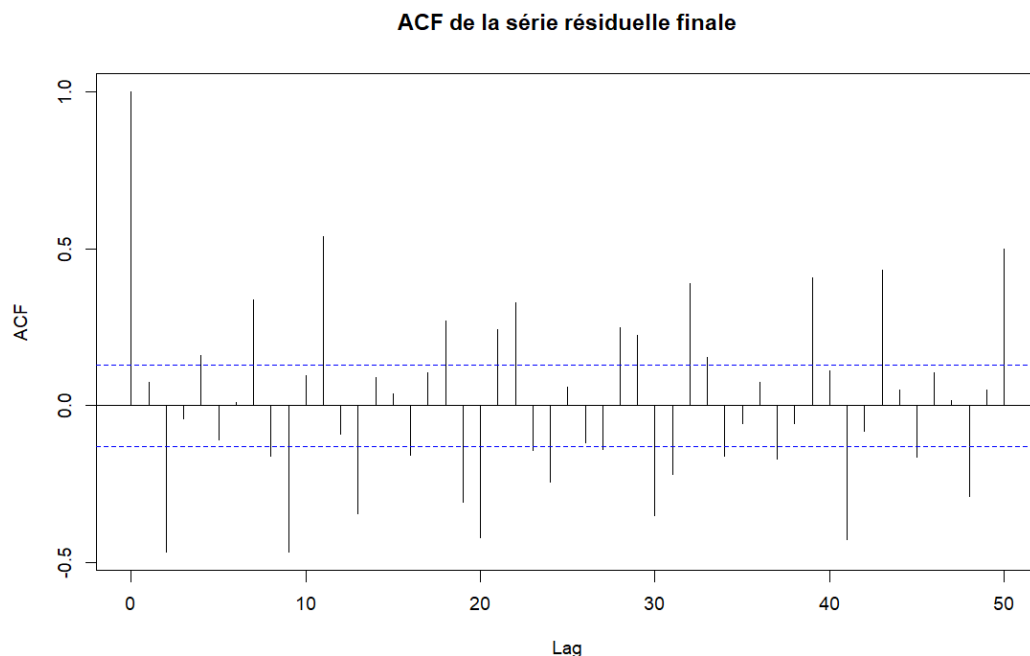
reg<-lm(detrended~fourier[,1:20]-1)
summary(reg)

ychap.lm.season<-xts(as.numeric(reg$fitted),order.by=monthly_avg$Date)
plot(detrended,type='l')
lines(ychap.lm.season,col='red', lwd=2)

```

Pour les données dont les cycles saisonniers sont insignifiants, il est préférable de simuler avec la méthode de Fourier, qui présente déjà une forte périodicité, car les fonctions de base sont toutes trigonométriques. Les deux premières méthodes sont davantage axées sur les études numériques.

Nous pouvons enfin tester nos résultats à l'aide d'un autocorrélogramme :



```

resid_final <- detrended - ychap.lm.season
resid_final_filled <- na.approx(resid_final)
plot(resid_final_filled, type = "l", main = "Série résiduelle finale",
     xlab = "Temps", ylab = "Résidus")

acf(as.numeric(resid_final_filled), lag.max = 50, main = "ACF de la série
résiduelle finale")

```

Comme le montre la figure, la corrélation des résidus a beaucoup diminué, mais il reste encore quelques informations saisonnières. Nous souhaitons l'analyser à l'aide de la fonction STL, cependant, les résultats montrent que le résidu n'est pas périodique ou que sa période est inférieure ou égale à 2.

4.4 La STL function

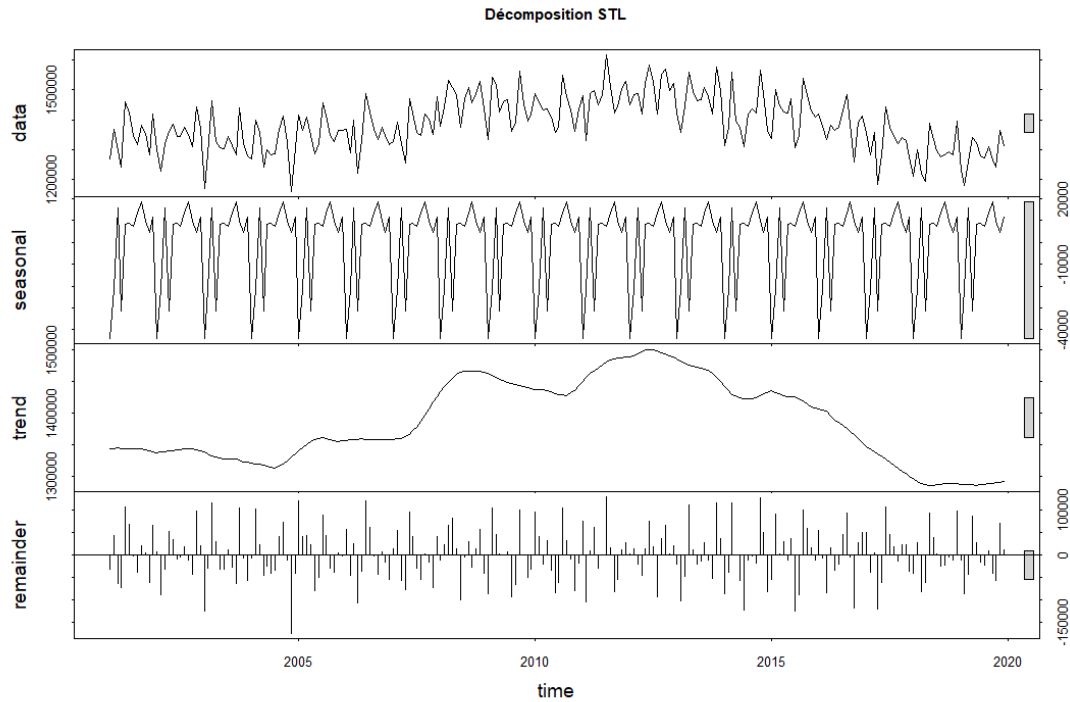
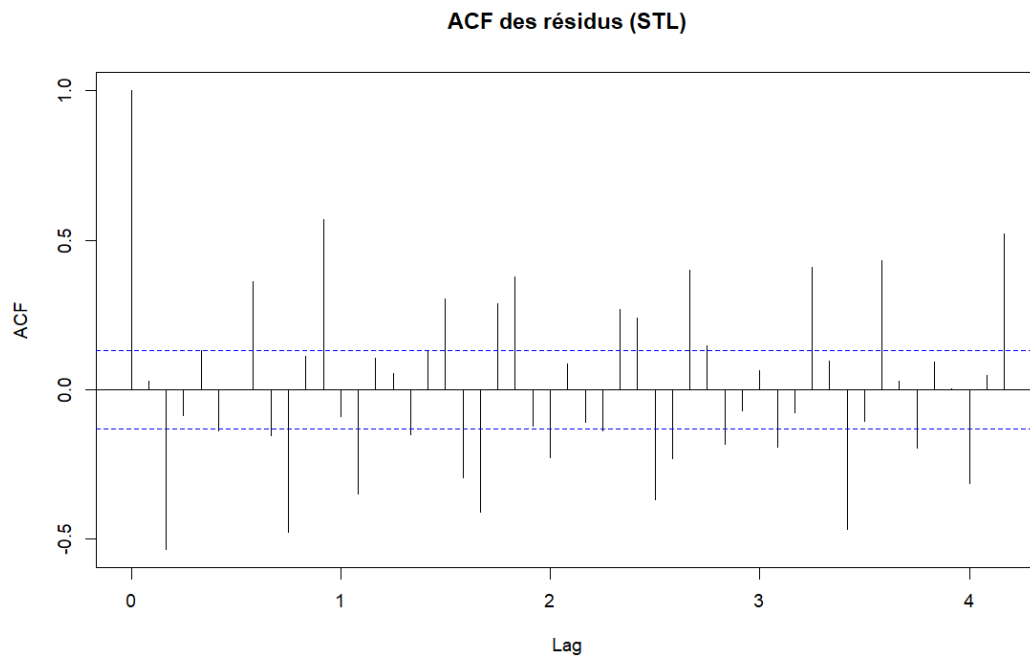


Figure 13: STL

```
stl_decomp <- stl(monthly_ts, s.window = "periodic")  
plot(stl_decomp, main = "D composition STL")
```

La comparaison révèle que la méthode STL analyse les tendances de manière plus heurtée, ce qui aurait pour conséquence que les résidus contiennent davantage de tendances saisonnières. En outre, la méthode produit des tendances saisonnières très favorables, mais nous n'en connaissons pas la raison.



En compensation, les termes résiduels sont plus fortement corrélés qu'auparavant.

5 Lissage et prévision

5.1 Lissage

5.1.1 Lissage exponentiel simple et double

Le lissage est réalisé manuellement avec les fonctions suivantes :

```
# Fonctions de lissage exponentiel (restent inchangées)
expSmooth <- function(x, alpha) {
  xs <- numeric(length(x))
  xs[1] <- x[1]
  for(i in 2:length(x)) {
    xs[i] <- (1 - alpha) * xs[i-1] + alpha * x[i]
  }
  xs
}

DoubleExpSmooth <- function(x, alpha) {
  n <- length(x)
  l <- numeric(n)
  b <- numeric(n)
  smooth <- numeric(n)
  l[1] <- x[1]
  b[1] <- if(n > 1) x[2] - x[1] else 0
  smooth[1] <- x[1]
  for(i in 2:n) {
    l[i] <- smooth[i-1] + (1 - (1 - alpha)^2) * (x[i] - smooth[i-1])
    b[i] <- b[i-1] + alpha^2 * (x[i] - smooth[i-1])
    smooth[i] <- l[i] + b[i]
  }
  list(smooth = smooth, l = l, b = b)
}
```

On obtient les résultats suivants (le choix de α est justifié après) :

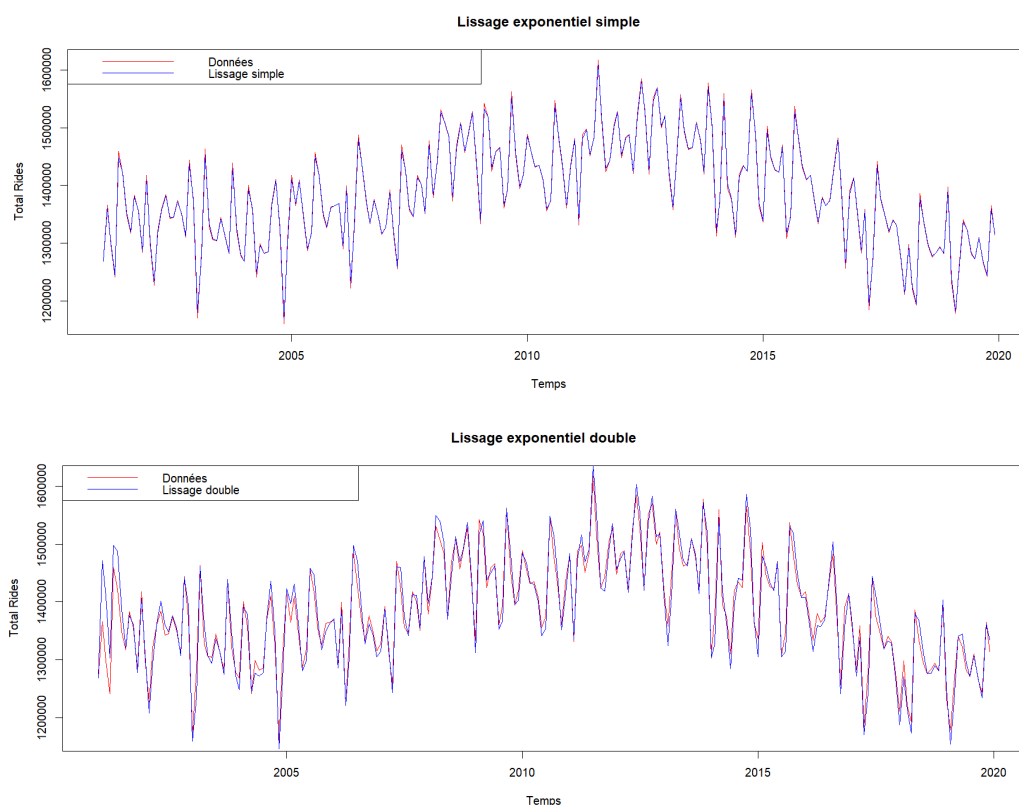


Figure 14: Lissage simple (haut) et double (bas)

Pour le choix du α , on peut calculer les résidus pour différentes valeurs de α et représenter cela :

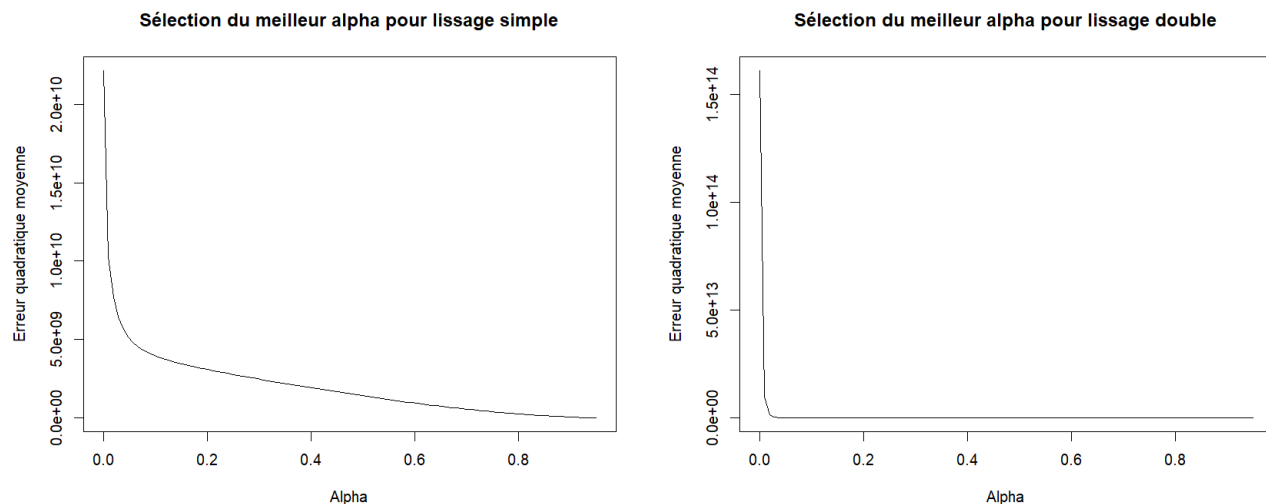


Figure 15: Choix de α (0.54 pour le lissage simple et 0.95 pour le lissage double)

5.2 Prédiction

La prédiction par lissage exponentiel double donne :

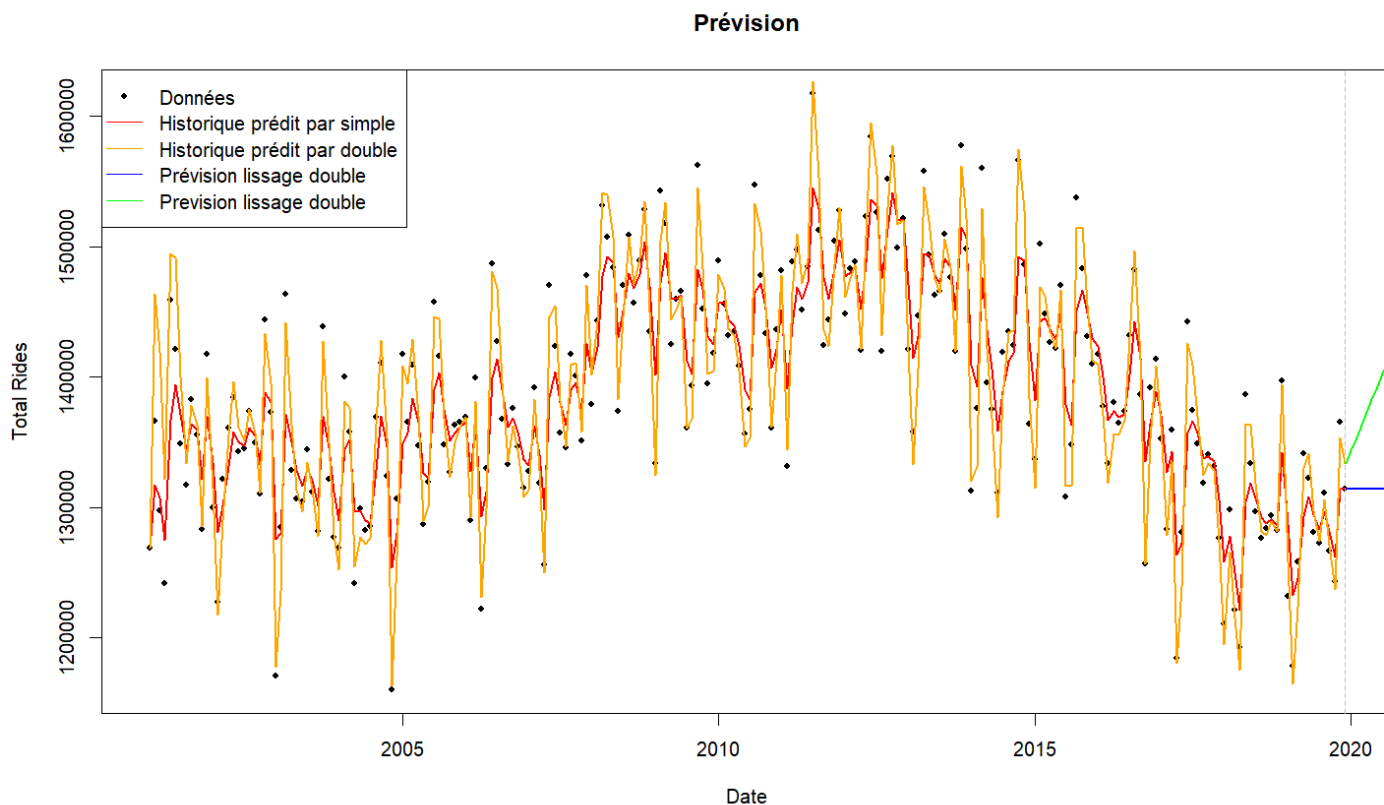


Figure 16: Prédiction

La prédiction 12 mois après 2020 s'élève avec un lissage simple à 1314026 voyageurs tandis que la prédiction par un lissage double donne 1442073 voyageurs.

Le modèle simple réalise une prédiction constante à partir de la dernière valeur.

Le modèle double réalise une prédiction d'équation : $prevision = l_{inst} + b_{inst} \times t$ où l et b sont les paramètres du modèle de lissage double.

Pour la validation nous avons décidé d'adopter une stratégie de *cross-block-validation*.

5.3 Validation croisée par blocs

Dans le code nous avons implémenté une validation croisée par blocs. Ceci consiste à découper les données par blocs, à effectuer une prévision sur le bloc à l'aide des autres données puis à calculer le résidu quadratique.

Les résultats ont été les suivants avec les différentes méthodes :

Méthode	MSE	$\sqrt{MSE}$	Erreur relative
Lissage exponentiel simple	11278278443	106199	7%
Lissage exponentiel double	12832305128	1386809	8%
Lissage exponentiel Hot-Winters	20130062609	141880	10%

Table 2: Résultats obtenus pour différentes prévisions avec $\alpha = 0.5$

À première vue le MSE peut paraître important, mais l'échelle des données l'est aussi, si bien que ramené à la moyenne, les erreurs relatives deviennent assez faibles.

Le lissage exponentiel double permet de prendre en compte la tendance dans le calcul de la prévision, la méthode Hot-Winters gère quant à elle la tendance et la saisonnalité (additive ou multiplicative). C'est finalement le lissage simple qui donne le meilleur résultat, c'est-à-dire que la prise en compte de la tendance et de la saisonnalité n'est pas significative dans notre modèle, même si les trois donnent des résultats assez proches.

À noter qu'il existe aussi d'autres méthodes de validations tels que les blocs glissants, et les erreurs relatives restent très proches. Les modèles sont globalement satisfaisant pour fournir une prévision.

6 Conclusion

Finalement c'est le lissage exponentiel simple qui fournit le meilleur résultat pour $\alpha = 0.5$, c'est-à-dire que l'on ne prend pas en compte la tendance ni la saisonnalité dans notre modèle qui ne sont pas significatives, après nos études.

Dans 12 mois le nombre de voyageurs sur le réseau de transport en commun de Chicago s'élèvera avec un paramètre α optimisé à 1314026.

7 Appendice

Initialement, nous avons travaillé avec des données de la RATP, mais celle-ci ne comportait pas assez de valeurs, car elles étaient en trimestres, c'est pourquoi nous avons opté pour des données journalière que nous avons moyennées par mois.